

Track - C
Computer Science

Part-1

Foundations for AI PM

LET'S DIVE IN ▶▶▶

Table of Contents

- **C1.1 Why a PM should learn CS foundations**
- **C1.2 Data Structures**
 - **C1.2.0 Big-O Notation**
 - **C1.2.1 Array / Contiguous Memory**
 - **C1.2.2 Linked List**
 - **C1.2.3 Stack**
 - **C1.2.4 Queue**
 - **C1.2.5 Hash Table**
 - **C1.2.6 Tree**
 - **C1.2.7 Graph**
 - **C1.2.8 Heap / Priority Queue**
 - **C1.2.9 Cheat card**

- **C1.3 Algorithms**
 - **Sorting**
 - **Searching**
 - **BFS/DFS**
 - **Dynamic Programming**
 - **Greedy**
- **C1.4 Complexity: Big-O as the PM's power tool**
 - **Notations · The ladder · Attention is $O(n^2)$ · Brute-force vector search · Complexity → latency → cost**
- **C1.5 The engineering negotiation**
- **C1.6 Interview Prep**

You will not implement these. You will reason about, negotiate over, and price systems built on them.

1

Why a PM should learn CS foundations

Four things you will do in a room with engineers, none of which is writing code.

C1.1 · The framing

You are not being trained to write algorithms

1. Translate a requirement into a cost shape:

- “Instant search across 50M docs” is not a UX request.

2. Smell a scaling wall before it's hit:

- When something is $O(n^2)$ or $O(2^n)$ in a quantity that grows

3. Call the bluff in both directions:

- Sometimes “it won't scale” is real.
- Sometimes it is premature optimization costing three sprints for a problem you'll never have.

4. Price the system:

- Every cache, index, and batch is a money decision. Complexity literacy is cost literacy.

The mantra.

“What grows, and how does cost grow with it?” asked relentlessly, that question is 80% of the value of this track.

2

Data structures

Eight structures.

For each: what it costs, what it maps to in your AI stack, and the question it hands you.

Asymptotic Notations

Definition.

A mathematical shorthand used to describe how the execution time or memory usage of an algorithm changes as the input size grows toward infinity

The Three Main Types

Computer scientists use three primary notations to analyze algorithm performance:

- **Big O:** Represents the worst-case scenario or upper bound. It guarantees an algorithm will not take longer than this limit.
- **Big Omega:** Represents the best-case scenario or lower bound. It guarantees an algorithm will perform at least this well.
- **Big Theta:** Represents the average-case scenario or tight bound. It describes the exact growth rate when upper and lower bounds match.

Why “Big-O” and not Big Omega or Big Theta?

1. Worst-case Guarantees
2. Real-World Inputs
3. Mathematical Simplicity
4. Industry Standard

Big-O describes growth rate, not run time

Definition.

A mathematical way to describe how the time or memory, an algorithm requires, grows as the input size increases, in the worst-case scenarios. It focuses on the growth rate, not the exact execution time.

What does n represent?

n is the size of the input. Its meaning depends on the problem:

Searching an array: n = number of elements

Sorting a list: n = number of items to sort

Traversing a graph: n = number of vertices (m = edges)

Processing a string: n = number of characters

Matrix operations: n = rows / columns

```
Array = [5, 8, 2, 9, 1]
```

```
n = 5
```

C1.2.0

The common notations, on one card

Big-O	Name	Example
O(1)	Constant	Access array element by index
O(log n)	Logarithmic	Binary Search
O(n)	Linear	Scan every element
O(n log n)	Linearithmic	Merge Sort, Heap Sort
O(n²)	Quadratic	Bubble Sort, Nested loops
O(n³)	Cubic	Triple nested loops
O(2ⁿ)	Exponential	Recursive Fibonacci
O(n!)	Factorial	Generating all permutations

Read top to bottom as a danger gradient. Everything above the O(n²) line is comfortable; everything below it is a design conversation.

C1.2.0 · Worked

O(1) and O(n): the two you'll meet most

Example 1: O(1) Constant

```
arr[4]
```

No matter whether the array has 10 elements or 10 million, retrieving by index takes essentially the same number of operations.

```
n = 10          → 1 operation
n = 1,000       → 1 operation
n = 1,000,000   → 1 operation
```

Example 2: O(n) Linear

```
for x in arr:
    print(x)
```

You visit every element once. The work grows proportionally with n.

```
n = 5          → 5 operations
n = 100        → 100 operations
n = 1000       → 1000 operations
```

The tell.

If the loop count is tied to something that grows with your product, documents, users, tokens, you are in linear territory and the growth curve is now a product question.

C1.2.0 · Worked

$O(n^2)$ and $O(\log n)$: the danger and the escape

Example 3: $O(n^2)$ Quadratic

```
for i in arr:
    for j in arr:
        print(i, j)
```

The inner loop runs n times for each of the n outer iterations.

```
n = 5      → 25 operations
n = 100    → 10,000 operations
n = 1000   → 1,000,000 operations
```

Example 4: $O(\log n)$ Logarithmic

Binary search repeatedly halves the search space. For $n = 16$, only 4 steps.

```
n = 16
16
↓
8
↓
4
↓
2
↓
1
```

n	$\log_2(n)$
8	3
16	4
32	5
1,024	10
1,048,576	20

Even for over one million elements, only about 20 comparisons needed.

Hold these two side by side. At a million items quadratic is a trillion operations and logarithmic is twenty. Almost every architecture decision in this track is a move from the left column to the right.

C1.2.0

Why constants are ignored. And the intuition ladder

Why ignore constants?

Algorithm A → $5n$

Algorithm B → $100n$

Both are $O(n)$. As n becomes very large they grow at the same linear rate. Big-O describes the rate of growth, not the exact runtime.

In summary.

n is the input size; Big-O describes how time or space scales as n grows.

Intuition Searching for a name

$O(1)$

Open directly to the correct page.

$O(\log n)$

Open in the middle, eliminate half repeatedly.

$O(n)$

Read every page one by one.

$O(n^2)$

Compare every page with every other page.

$O(2^n)$

Try every possible combination.

$O(n!)$

Try every possible ordering.

The full ladder, with the Principal-level nuance about when NOT to optimize, is Part 5.

C1.2.1

Array / contiguous memory: tensors are arrays

What it is:

A block of memory holding elements back-to-back.

Address = base + index $\times$ element_size.

One multiply-add, no searching.

Mental model:

A row of numbered lockers.

Instant if you know the number; slow if you must open every one.

The AI hook.

Model weights, embeddings, image batches are all contiguous numerical arrays. GPU throughput depends on coalesced access. This is why deep learning uses dense arrays, not linked structures.

Row- vs column-major:

Iterating against the layout thrashes cache: 10 $\times$ slower for identical math.

PM question:

“Dense array or pointer-chasing?”

Dense = fast + memory-hungry.

100M $\times$ 1536, float32 $\approx$ 600 GB.

Explainer

Coalesced memory access and why contiguity is speed

Coalesced (fast)

```
Thread 0 → Address 100
Thread 1 → Address 101
Thread 2 → Address 102
...
Thread 31 → Address 131

32 Reads → 1 Memory Transaction
```

Non-coalesced (slow)

```
Thread 0 → Address 100
Thread 1 → Address 5600
Thread 2 → Address 42
Thread 3 → Address 100000
...

32 Reads → Many Transactions
```

Why pointer-chasing stalls:

A linked list lands at 100 → 9845 → 27 → 600001.

Every access depends on the last, addresses scatter, nothing coalesces.

The library analogy.

32 people fetching books.

Coalesced: all 32 on one shelf, one trip.

Non-coalesced: one on Floor 1, one on Floor 7, one in the Basement, many trips.

GPUs execute threads in groups called warps: typically 32 threads on NVIDIA hardware.

Explainer

Warps, SIMT, and warp divergence (NVIDIA)

A warp is the smallest unit of execution:

Launch 128 threads and the GPU schedules 4 warps of 32, not 128 independent threads.

All 32 run the same instruction on different data.

Threads:

```
0 ... 31    → Warp 0
32 ... 63    → Warp 1
64 ... 95    → Warp 2
96 ... 127   → Warp 3
```

SIMT — Single Instruction, Multiple Threads

```
C[i] = A[i] + B[i]
Thread 0 → C[0] = A[0] + B[0] ...
```

Warp divergence

```
if value > 0:
    do_A()
else:
    do_B()
```

```
Run do_A() for Threads 0-15
Threads 16-31 wait
```

Then

```
Run do_B() for Threads 16-31
Threads 0-15 wait
```

Why 32 matters. Scheduling, memory coalescing, and synchronization are all managed per warp.

Explainer

CPU vs GPU: two different bets

CPU	GPU
Designed for fast, sequential processing of diverse tasks	Designed for massive parallel processing of repetitive tasks
Executes a few powerful threads	Executes thousands of lightweight threads
Schedules individual threads	Schedules warps of 32 threads
Optimized for low latency	Optimized for high throughput
Handles different instructions efficiently	Most efficient when threads in a warp execute the same instructions

Interview-ready. “Why are GPUs fast for AI?”
Because the core operation is dense matrix multiplication over contiguous arrays, and GPUs pair thousands of cores with memory designed for coalesced access to adjacent addresses.

Explainer

Vector database sizing & where “~600 GB” comes from

Step 1 100M = 100,000,000 vectors

Step 2 1536 dimensions each
 $100,000,000 \times 1536$
= 153.6 billion floats

Step 3 float32 = 32 bits = 4 bytes

Step 4 153.6 billion $\times$ 4
= 614,400,000,000 bytes

Step 5 $\div 1,000,000,000$
= 614.4 GB $\approx$ 600 GB

The formula

$$\begin{aligned} \text{Memory} &= (\text{Number of vectors}) \\ &\times (\text{Dimensions}) \\ &\times (\text{Bytes per value}) \end{aligned}$$

Why a PM cares. That number alone dictates architecture. It is the difference between “this runs on one machine” and “this is a distributed systems project.”

Explainer

Quantization: buying the memory back

```
100M documents
  ↓
1536 numbers each
  ↓
4 bytes per number
  ↓
≈ 614 GB RAM
```

The trade. Reducing storage per dimension dramatically lowers memory requirements while often preserving acceptable search quality. A 32× memory cut for a small recall cost is one of the highest-leverage levers in the stack.

Ask engineering which precision the index uses before you ask how many machines it needs.

Linked list and Stack (LIFO)

	Linked List	Stack (LIFO)
What it is	Nodes scattered in memory, each holding a value plus a pointer to the next. No indexing ,you walk the chain. Access by position $O(n)$ · insert/delete given the node $O(1)$ · search $O(n)$.	Push and pop only from the top. $O(1)$ both.
Mental Model	A treasure hunt. Great for inserting a clue mid-hunt; terrible for “jump to clue #7,000.”	A stack of plates. You take the top one.
AI Hook	A sequence of tokens, events, conversation history, is a linked chain. Streaming token output is this pattern.	Recursive / nested agent execution. When an agent spins up a sub-task which spins up another, “return to the parent when the child finishes” is a stack. Depth-first agent planning uses one implicitly.
PM Conversation	Rarely direct, but the trade-off “random access vs cheap insertion” underlies a lot (“we can append new events cheaply, but computing ‘the 500th event’ is a scan”).	“What's the max nesting depth before this agent recursion blows up or loops forever?”

C1.2.4

Queue: How requests are drained IS the economics

What it is: Enqueue at the back, dequeue from the front.

O(1) both. Variants: priority queue, deque.

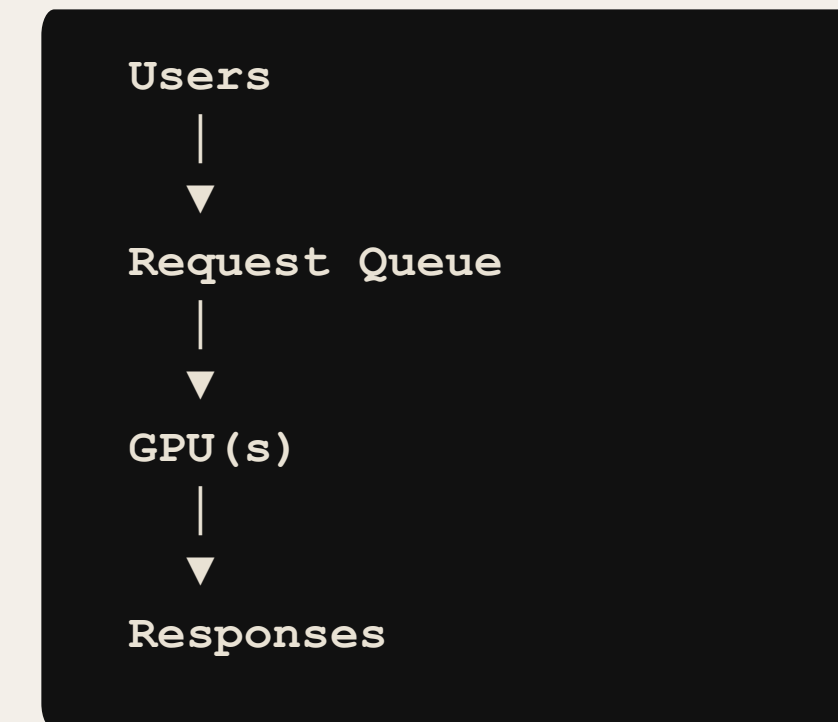
Mental model: A line at a coffee shop. First to arrive, first served. FIFO

Static batching: Collect N, run together, return together.

The whole batch waits for the longest generation.

Continuous batching: vLLM's key idea. Finished requests leave mid-flight, new ones join. Lifts GPU utilization 2–4×.

Message queues: Kafka, SQS, Celery sit between services, the backbone of any async AI pipeline.

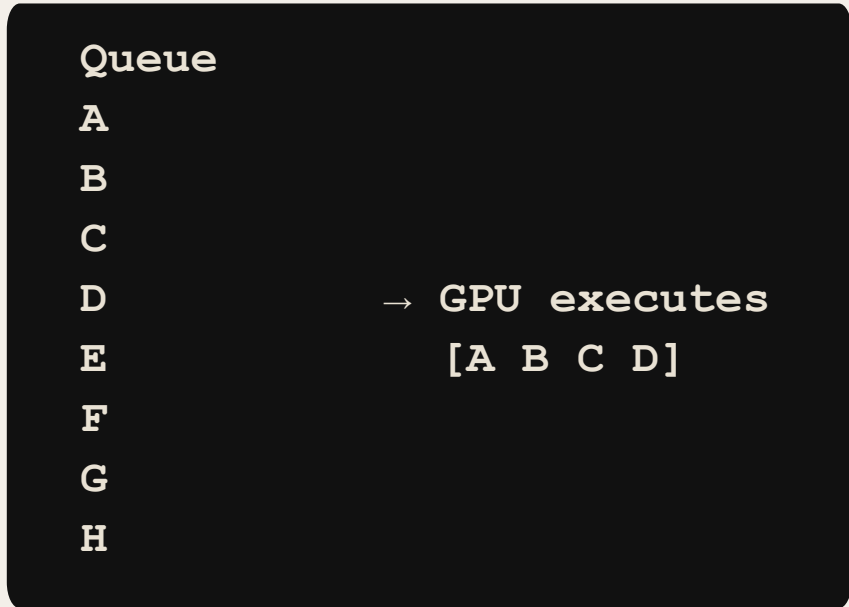


PM question. “At peak QPS, how deep does the queue get, and what's the p95 wait before generation even starts?”
Queue-wait is invisible in a demo and brutal in production.

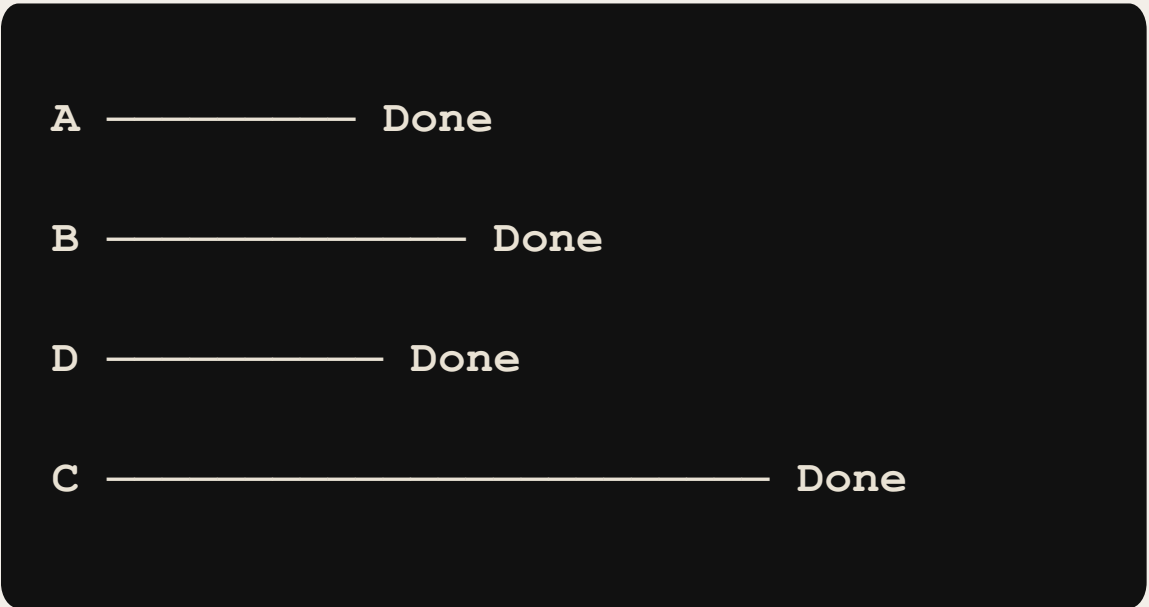
Explainer

Static batching and the straggler problem

Four requests, one batch



Request	_____	Tokens to Generate
A	_____	20
B	_____	30
C	_____	500
D	_____	25



A, B and D finish early, but must wait for C.

The cost:

- Higher latency for short requests
- GPU idle for completed slots
- Poor utilization.

This is the straggler problem. The GPU cannot return the batch until the longest request finishes, so one 500-token generation holds three 25-token replies hostage.

Explainer

Continuous batching: Refill the slot every step

Step 1	Step 2
A	A Smaller token req
B	B
C	C
D	D
Step 3	Step 4
A finishes	B
B	C
C	D
D	E
E joins	

Why token-by-token works:

LLMs generate one token at a time.

Every step the engine asks: **which requests still need the next token?**

E did not wait for D: That single fact is the whole idea.

Why this saves money. The GPU costs the same. Continuous batching lifts utilization from ~40% to 80–95%, so the same hardware serves far more requests and each request becomes much cheaper.

This is why serving architecture directly affects cost per token, a line item, not an implementation detail.

Explainer

vLLM

Earlier → fixed batches.

vLLM → highly efficient continuous batching.

vLLM became popular with:

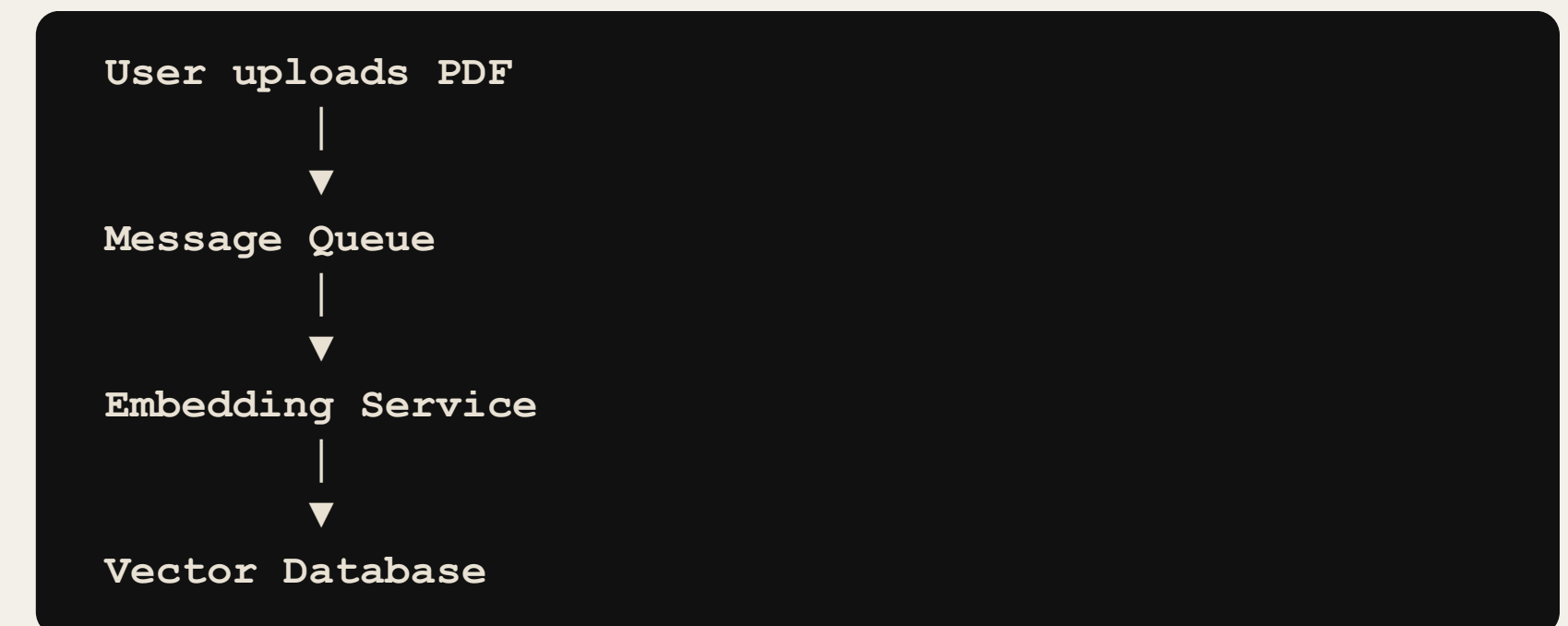
- Higher GPU utilization
- Lower latency
- More concurrent users
- Lower cost per generated token

The headline. For many deployments, changing only the serving layer yields 2–4× higher throughput without changing the model.

Used for embedding generation, evaluation jobs, data ingestion, and model retraining.

The message queues behind the pipeline

Message queues decouple the services



Without a queue: If embedding is slow or crashes, uploads are delayed or fail.

With a queue: Upload completes immediately; workers process independently.

Examples: Kafka, SQS (AWS Simple Queue Service), Celery.

Explainer

Queue depth: the early warning of overload

Can the product sustain real production traffic?

Setup: 4 GPUs · each processes 25 requests/sec · total capacity 100 QPS. Then traffic spikes.

Time	Incoming QPS	Capacity	Queue

10:00:00	80	100	0
10:00:10	120	100	200
10:00:20	150	100	1200
10:00:30	180	100	3600

Read the last column. The queue keeps growing because requests arrive faster than GPUs can drain them.

The PM question:

At peak traffic, does the queue reach 20 requests?
2,000? 50,000?

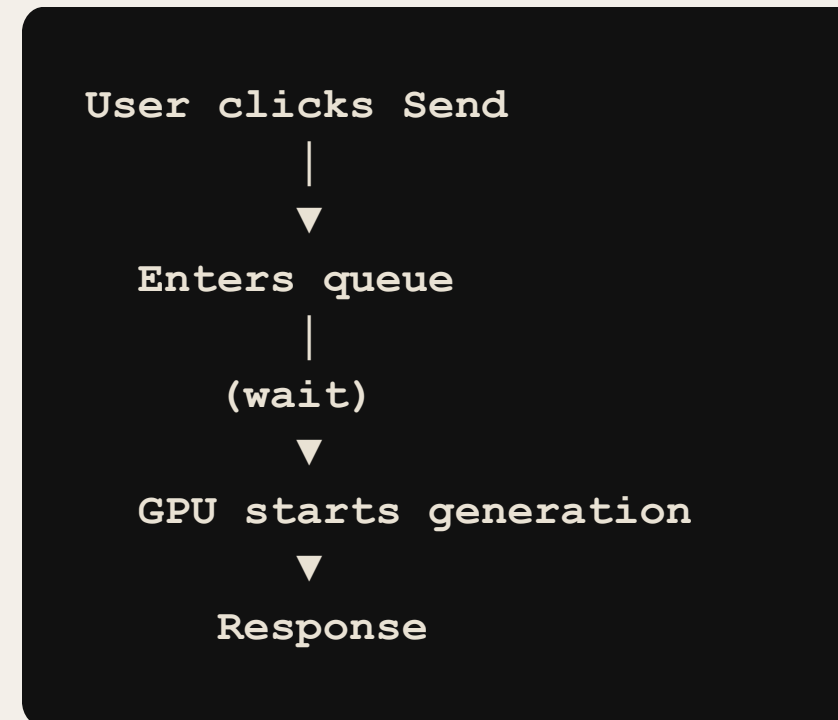
Queue depth is the early warning that the system is overloaded, well before anyone files a latency bug.

Explainer

p95 queue wait: the latency your dashboard hides

P95:

The 95% percentile (95% of total users) used to measure system latency of response times.



```
90 requests → 50 ms
5 requests → 300 ms
5 requests → 3 seconds

Average ≈ 220 ms
p95 ≈ 3 s
```

Why the demo lied. One user means queue wait = 0 ms. Production at 5,000 concurrent users means queue wait 2.8 s and generation 700 ms. Users say “the AI is slow.” The model isn't slow, the queue is.

Stop asking “how fast is the model?”:

Ask instead: peak QPS · concurrent requests per GPU · average and p95 queue wait · how often we breach SLO · autoscale or throttle · is queue wait or generation the dominant term?

Explainer

SLI, SLO, SLA: the vocabulary of targets

SLA	SLO	SLI
Service Level Agreement	Service Level Objective	Service Level Indicator
A contractual commitment, often with penalties if missed	The Target	The measured metric
99.9% uptime guaranteed to customers	p95 latency < 2s	actual p95 latency = 1.8s

Typical SLOs

- Latency p95 < 2 s
- Availability 99.9%
- Queue wait p95 < 200 ms
- Error rate < 0.1%

Worked example.
SLI: measured p95 queue wait = 180 ms.
SLO: p95 queue wait should be < 200 ms.
Result: the SLO is being met.

C1.2.5

Hash table: the structure behind every cache

What it is: Map keys to values by running the key through a hash function to compute a slot. Average $O(1)$ insert / lookup / delete; worst case $O(n)$ on heavy collision.

Mental model: A coat check. You don't search the racks, the ticket number sends you straight to the hook.

KV cache: Reuses computed Key/Value tensors for past tokens instead of recomputing.

Prompt / response caching: Turns a paid model call into a free dictionary hit.

Feature stores: `user_id` $\rightarrow$ features at inference, hash-backed.

Deduplication: Training-data cleaning, near-dup detection via MinHash.

PM question. “What's our cache hit rate, and what does each hit save in latency and dollars?” A 40% exact-match hit rate on an expensive model is a line item, not a detail.

Hash tables are everywhere in AI infra

This is why the first token is slow (prefill) and later tokens are fast, and why long conversations eat GPU memory.

Explainer

Hash tables

```
"Alice"    → 95
"Bob"      → 88
"Charlie"  → 91
```

The coat check: You hand over your coat and get ticket #257. Later you show #257 and staff find it instantly. They don't search thousands of coats. The ticket number tells them exactly where to look.

The hash function: Converts a key into a number. "Alice" → hash → Slot 17.

Without a hash table

```
Search 1
Search 2
Search 3
...
Search 10,000
```

With a hash table

```
Key
↓
Hash Function
↓
Correct Slot
```

The payoff. Average lookup is $O(1)$, almost equally fast at ten entries or ten million.

Explainer

Where hash tables show up

1. **KV Cache:** While generating “Hello, how are you”, the model remembers calculations for previous words rather than recomputing. Token position → cached computation. First token slow, later tokens fast.
2. **Prompt Caching:** 10,000 users ask “What is Newton's First Law?” Hash the prompt, return the stored answer. No expensive model call — saves time, GPU, money.
3. **Feature Stores:** User ID 12345 → age, purchase history, preferences, subscription. Keyed lookup at inference time.
4. **Deduplication:** Training data containing Article A three times, hashing detects duplicates so the same document isn't processed repeatedly.

This makes caching a product and business optimization, not just an engineering detail.

The PM arithmetic

```
100 requests
 40 served from cache
 60 sent to the LLM
```

```
Cache Hit Rate = 40%
```

```
40 × $0.05 = $2 saved
```

A higher hit rate means:

- Faster responses
- lower GPU usage
- lower infrastructure cost
- better user experience.

C1.2.6

Tree: hierarchy that buys $O(\log n)$

What it is: Hierarchical nodes, one root, no cycles. Balanced search trees give $O(\log n)$ search / insert / delete.

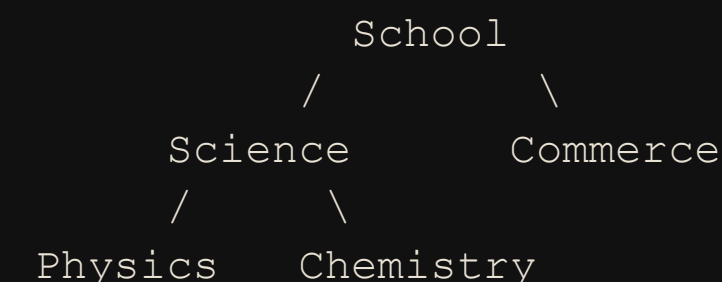
Mental model: An org chart, a family tree, a folder hierarchy.

Tokenizer merge trees: BPE builds subword vocab via merge rules, why token counts $\neq$ character counts.

Decision trees & GBTs: XGBoost / LightGBM still the best default for tabular prediction.

Tree-of-Thought: An LLM explores a tree of candidate reasoning paths and prunes.

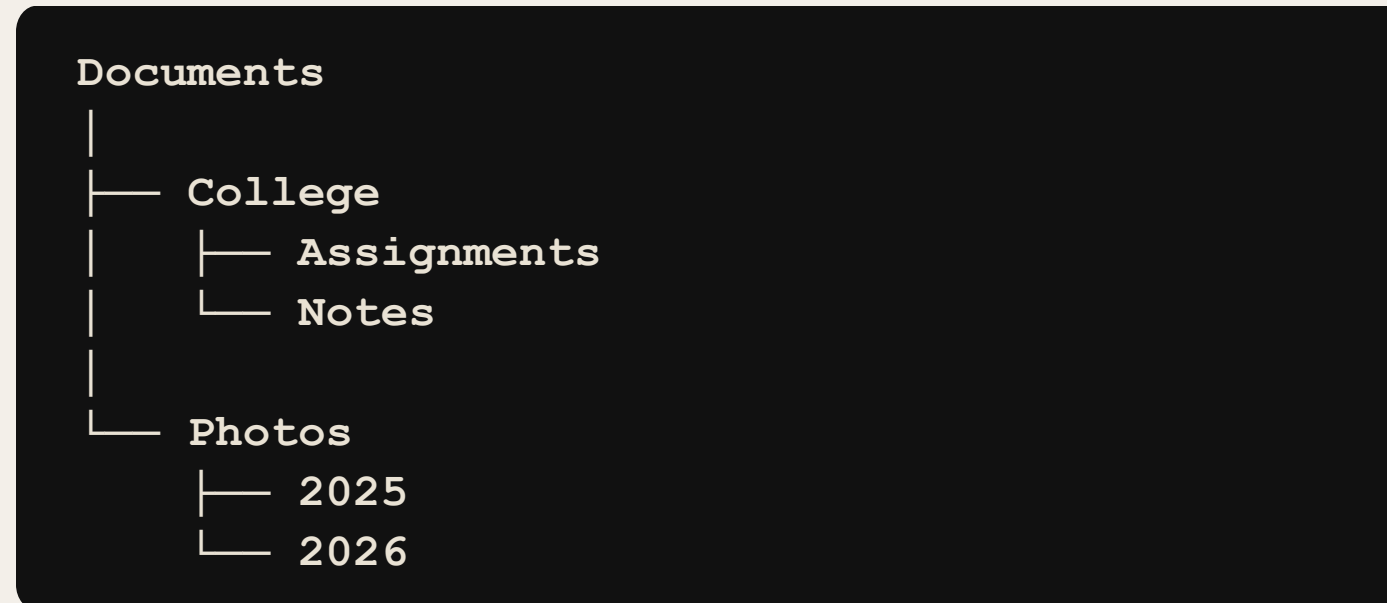
ANN indexes & ASTs: Annoy and k-d trees; abstract syntax trees for code models; B-trees under your database.



PM question. “Is retrieval tree-based or graph-based, and what's the recall / latency trade at our scale?” The vocabulary starts here; the decision lands in Week 8.

Explainer

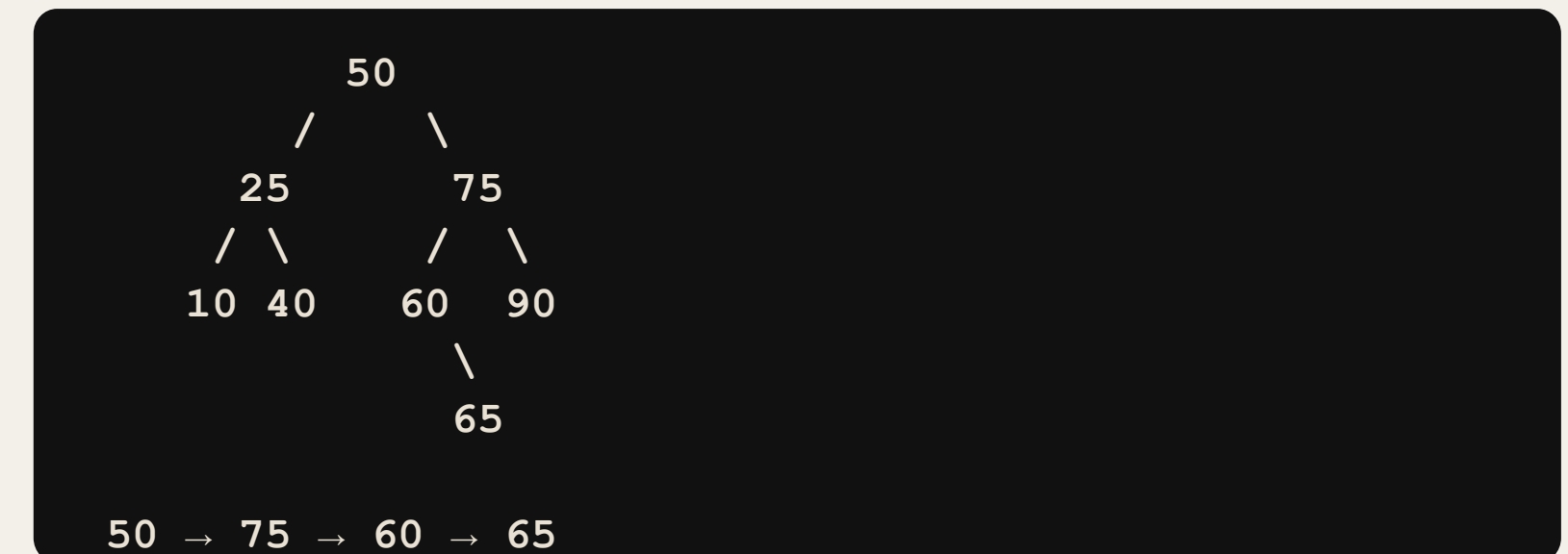
Trees (contd)



Every node has one parent (except the root) and zero or more children. Unlike a graph, a tree has no cycles.

Think of. A family tree, an org chart, the folder structure on your laptop.

Finding 65 is four comparisons, not many



The mechanism. A balanced search tree eliminates half the remaining possibilities at each step. That gives $O(\log n)$.

Explainer

Six places trees run your AI stack

1. **Tokenizers:** “unbelievable” → un / believ / able. Pricing is per token, not per character — two words of equal length can cost very differently.
2. **Decision Trees:** Income > ₹10L? → Credit > 700? → Approve. Best-in-class for structured data.
3. **Tree of Thoughts:** The LLM explores several reasoning branches and keeps the best, like trying multiple routes through a maze.
4. **Tree-based search:** With 100M embeddings, the tree narrows down where similar vectors are likely to be.
5. **Abstract Syntax Trees:** $x = a + b * c$ becomes a tree the model reads to understand expressions and control flow.
6. **B-Trees in databases:** `SELECT * FROM Users WHERE user_id = 12345` doesn't scan every row, it uses a B-Tree index.

Tree vs graph retrieval. Tree-based: faster, less memory, may miss some neighbours. Graph-based (HNSW): higher recall, more memory, slightly higher latency. Lowest latency → tree. Highest quality for RAG → graph.

Explainer

BPE: how tokenization actually works

A tokenization algorithm that splits text into tokens before the model processes it. A token may be a whole word, part of a word, punctuation, or a space.

How BPE builds a vocabulary

```
Step 1  start with characters  
l o w / l o w e r  
n e w / n e w e s t
```

```
Step 2  merge commonest pair  
e + r → er
```

```
Step 3  repeat  
l + o → lo  
lo + w → low  
low + er → lower
```

Why it's needed. Computers understand numbers, not words. Before an LLM reads “unbelievable” it converts it to un + believ + able.

Example

```
Vocabulary already contains:  
play / ing / player
```

```
So:  playing → play + ing
```

```
Not: p l a y i n g
```

```
Fewer tokens are needed.
```


Why tokenization is a pricing question

```
happy  
happily  
happiness  
unhappy
```

```
reuse: happy / ness / ly / un
```

English has millions of possible words. Instead of storing every word, BPE stores common pieces, dramatically reducing vocabulary size while still representing almost any word.

Why “byte pair”? The original algorithm compressed files by repeatedly replacing the most common pair of bytes with a new symbol. LLM tokenizers adapted the idea, merge, repeat thousands of times, and land on a vocabulary of roughly 30,000–100,000 tokens.

The consequence. LLMs charge by tokens, not words. One long word may cost many times more than a short one.

C1.2.7

Graph: nodes, edges, and the agent question

What it is: Nodes (vertices) plus edges, possibly directed, possibly weighted. Can have cycles. Stored as adjacency lists or matrices.

Mental model: A subway map, a social network, a citation web.

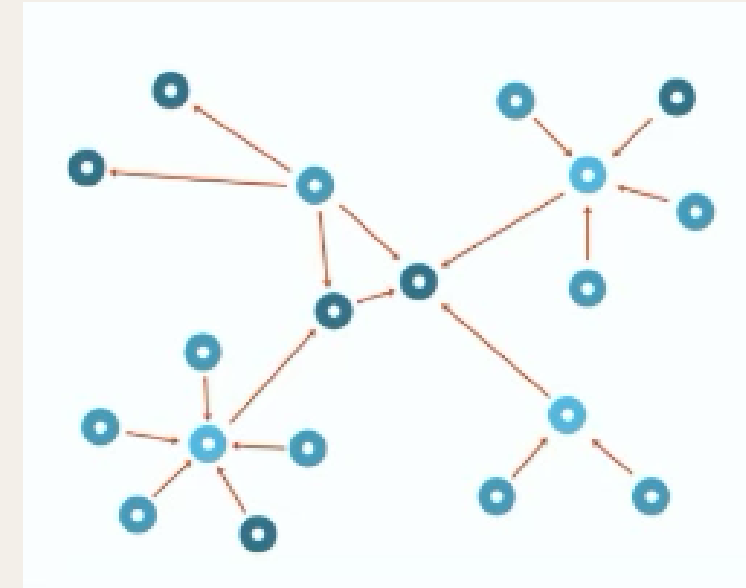
Knowledge graphs & GraphRAG: Retrieval traverses relationships instead of just matching vectors, better for multi-hop questions.

Agent workflow engines: LangGraph is literally a state graph. When you design an agent, you are drawing a graph.

HNSW: A multi-layer graph you greedily walk in $\sim O(\log n)$.

Why semantic search over 100M vectors in 20 ms is possible!!

Computation graphs: How PyTorch and TF represent a model for autodiff.



The architecture call. “Is this a workflow or an agent, where the model chooses the path?” Deterministic graph vs learned traversal is one of the most important decisions you'll make.

Used by: Pinecone · Weaviate · pgvector · FAISS ·
LangGraph · Airflow · Prefect · Ray

C1.2.8

Heap / priority queue: every top-k you ship

What it is: A tree-shaped structure keeping the min (or max) instantly reachable. Peek $O(1)$; insert and pop $O(\log n)$.

The number that matters: Top-k of n items is $O(n \log k)$, much cheaper than fully sorting at $O(n \log n)$.

Mental model: Hospital triage. You always pull the most urgent patient next, without sorting the whole waiting room.

Also: Task schedulers (run the highest-priority job next) · beam search (keep top-k beams) · Dijkstra and A* for agent path planning.

The AI hook. Top-k retrieval is a heap operation. Every RAG retrieval, every recommendation “top N”, every re-ranker shortlist is a top-k.

PM question. “What's k , and is k tuned for recall or cost?” Bigger k = more context to the LLM = better recall but more tokens, more dollars, more latency. k is a dial you own.

C1.2.9

The cheat card... screenshot this one...

Structure	Access	Search	Insert	Delete	The AI thing to remember
Array	$O(1)$	$O(n)$	$O(n)^*$	$O(n)$	Tensors; GPU loves contiguity
Linked list	$O(n)$	$O(n)$	$O(1)^{\dagger}$	$O(1)^{\dagger}$	Sequences; streaming tokens
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$	Recursion; agent sub-tasks; undo
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	Inference batching; service decoupling
Hash table	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	KV cache; prompt cache; feature store
Tree (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Tokenizer/BPE; GBTs; ToT; Annoy
Graph	—	varies	$O(1)^{\ddagger}$	varies	Knowledge graph/GraphRAG; LangGraph; HNSW
Heap	$O(1)$ peek	$O(n)$	$O(\log n)$	$O(\log n)$	Top-k retrieval; schedulers; beam search

** append $O(1)$ amortized · $\dagger$ given the node/position · $\ddagger$ add edge/node*

3

Algorithms

Five families.

Each one shows up in your product as a latency line, a bill, or a quality ceiling.

C1.3 · Algorithms

Algorithms: Deep Dive with AI Decoding Connections

Sorting

Searching

BFS / DFS

Dyn. Prog.

Greedy

Five algorithm families — each one quietly running inside an AI system you already use.

- Sorting: powers ranking pipelines and re-rankers: every search result and recommendation feed is a sort.
- Searching: the index bargain behind every vector database: pay once to organize, then every query is fast.
- BFS / DFS: how agents plan and how knowledge graphs are queried: explore wide or go deep, each is a real design axis.
- Dynamic Programming: the KV cache and every caching win: solve once, store, and never recompute the same subproblem.
- Greedy: how an LLM picks its next token: always take the locally best option, and understand when that is not enough.

Running question across all five families:
what does this cost, and who decides the bound?

C1.3 Algorithms

Sorting

Sorting is arranging a collection into a defined order.

Its is the cheapest way to buy structure and structure is what makes every later operation fast.

Two things a PM should hold onto:

- Sorting has a provable speed limit. You cannot beat it, no matter how clever your engineer is.
- Sorting is often more work than you actually need. Knowing when to skip it is where the money is.

The $O(n \log n)$ Floor

- No comparison-based sort can beat $O(n \log n)$. This isn't "nobody has found one yet", it's proven.
- Python's sorted() uses Timsort, which is $O(n \log n)$ worst case
- The recurring shape is always: score every candidate → sort by score → take the top-k.

Where Sorting lives in AI:

Surface	The sort
Search results	Rank documents by relevance score
Recommendation feeds	Rank items by predicted engagement
Re-rankers	Reorder a retrieved shortlist with a heavier model
Leaderboards / eval scoring	Rank model variants by metric

Ranking Pipelines and the Top-k Trap

The Top-k Trap:

- If you only need the top-k results, do not fully sort.
- Full sort of 1M candidates to display 10 is pure waste: you computed 999,990 unnecessary comparisons.

Better options:

- Use a min-heap at $O(n \log k)$
- Use `np.argpartition` at $O(n)$: `np.argpartition` guarantees the k smallest or largest elements are in place, unsorted among themselves, but that is fine.

Spotting this in a design review is a PM-level move.

Why this matters at scale. Sorting 1,000,000 candidates fully costs ~20M operations. A heap with $k=10$ costs ~3M. `np.argpartition` costs ~1M. The gap compounds across millions of daily queries — the right algorithm choice here directly reduces infrastructure spend.

The question to ask in any ranking system: do we need a full sorted list, or just the top-k? The answer changes the algorithm, the cost, and the latency.

C1.3 Algorithms

Searching

Searching means finding one item among many.

Linear Search: $O(n)$

Given the list 12, 45, 7, 90, 31, find 90.

Check each element in order: 12 (no), 45 (no), 7 (no), 90 (yes).

Worst case: searching for 31 means examining every single element in the list.

```
for item in data:
    if item == target:
        return item
```

Check every element one by one. Works on any data, sorted or not.

With 1 million items you may perform 1 million comparisons.

```
n = 1,000      → 1,000 comparisons
n = 1,000,000 → 1,000,000 comparisons
n = 1,000,000,000 → 1B comparisons
```

Binary Search: $O(\log n)$

Given the sorted list 7, 12, 31, 45, 90, find 45.

Do not start at the beginning; look in the middle. 31 is too small, so discard the entire left half. Only 45 and 90 remain, found in just 2 comparisons.

```
import bisect
idx = bisect.bisect_left(sorted_data, target)
```

Requires sorted data.

Each comparison halves the search space, 1M items needs only ~20 comparisons.

```
n = 1,000      → ~10 comparisons
n = 1,000,000 → ~20 comparisons
n = 1,000,000,000 → ~30 comparisons
```

C1.3 Algorithms

Linear Search $O(n)$

1 Time complexity: $O(n)$

With 1 million items you may perform 1 million comparisons. Every element must be checked until the target is found — or confirmed absent.

2 The upside of linear search

It works even if the data is completely unsorted. No preprocessing required. Any list, any order — just check each element one by one until found.

*Linear search is the baseline.
It always works, but structure buys speed.*

The canonical trade.

Pay a one-time cost to sort and impose structure so every future query is cheap. Python's bisect module is binary search, use it in the lab. Vector indexes apply this exact bargain at scale: build structure once, search fast forever.

Binary Search $O(\log n)$

1 Check the middle element

List: 7, 12, 31, 45, 90. Middle = 31. Target is 45. Since $45 > 31$, discard the entire left half — 7, 12, and 31 are gone in one step.

2 Find the target in the remainder

Only 45 and 90 remain. Check the middle: 45 matches. Found in 2 comparisons total. The catch: this only works on sorted data — on unsorted data you cannot safely discard any half.

Every comparison eliminates half the remaining candidates, that is the power of $O(\log n)$.

C1.3 Algorithms: Search

Why Halving Is So Fast

Every comparison cuts the search space in half

1,000,000 becomes 500,000, then 250,000, then 125,000, and keeps halving

~20 halvings reach one item from a million

After about 20 steps, only a single item remains, the search is done

1,000 items — roughly 10 comparisons

1,000,000 items — roughly 20 comparisons

1,000,000,000 items — roughly 30 comparisons

A billion items adds only ten more steps, the cost barely grows

What $O(\log n)$ means in practice

Doubling the data adds just one comparison, logarithmic growth is almost flat

C1.3 Algorithms: Search

The Library Example

One-time organization cost, faster forever

Without Organization:

- Books scattered randomly on shelves
- Finding one book means checking shelf after shelf
- Every visitor searches the same slow way → Linear Search $O(n)$

With Organization:

- Books arranged alphabetically
- Go straight to the correct section
- Every visitor finds books much faster from day one → Binary Search $O(\log n)$

The library spent time organizing once and now every visitor benefits.

The principle:

Spend time organizing data once so every future search is fast.

One-time cost → permanent gain.

The canonical trade in all of AI infrastructure:

Vector indexes, search engines, and databases all pay an upfront organization cost so that every query is cheap. No structure means every search is $O(n)$; build structure first and searches approach $O(\log n)$.

C1.3 Algorithms: Search

From Binary Search to Vector Indexes: Build Once, Search Fast

Step 1 → No Index

100M embeddings

Query arrives, Compare all 100M

$O(n)$ every time, Extremely slow

Step 2 → Build Index

Organize embeddings

Group by proximity, Pay cost once

Structure stored, Ready to serve

Step 3 → Query w/ Index

Query arrives

Search small subset, Skip most vectors

Near $O(\log n)$, Result returned fast

Why vector indexes exist:

Brute force compares every embedding — at 100M vectors that is 100,000,000 dot products per query.

ANN (Approximate Nearest Neighbor) indexes organize vectors before users search:

- Build the index once (expensive)
- Each query searches only a tiny fraction
- Approximate Nearest Neighbor trades tiny accuracy loss for massive speed gain

The index acts like the sorted library:

Spend time organizing data once so every future search is fast.

No structure → every search is $O(n)$

Build structure first → searches approach $O(\log n)$

Why this matters for RAG pipelines. Every RAG system, search engine, recommendation feed, and vector database builds an index before serving queries. The build cost is a one-time infrastructure investment; the query savings are paid back on every single user request, at scale, this is the difference between a viable product and an unaffordable one.

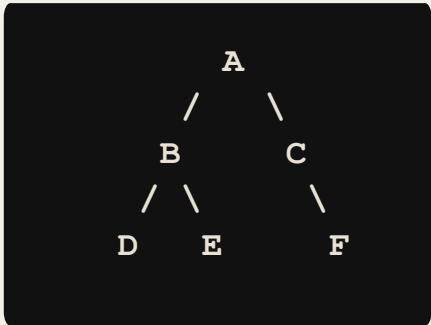
This is why vector databases like Pinecone, Weaviate, and pgvector exist: the index is the product, not the vectors.

C1.3 Algorithms

BFS vs DFS: Graph Traversal

How do we find the right element in a Graph?

A graph is a collection of nodes connected by edges



AI connections:

- Agent planning:
 - BFS = many plans shallow;
 - DFS = one plan deep
- Multi-hop RAG: A→B→C across a knowledge graph
- Tree-of-Thought: graph of possible reasoning paths

Two strategies, one graph

BFS (Breadth-First)	DFS (Depth-First)
Explores level by level	Follows one path completely
Uses a Queue (FIFO)	Uses a Stack (LIFO)
Finds shortest path by hops	Does NOT guarantee shortest path
Uses MORE memory, since it keeps all nodes in memory	Uses LESS memory, since only current path is stored
Better for nearby answers	Better for deep exploration
Order: A, B, C, D, E, F	Order: A, B, D, E, C, F

The PM decides: Every traversal step costs GPU time, latency, money and safety. So, how deep or wide can the agent search before we stop? How deep can the agent go with safe search limits preventing infinite lops or runaway reasoning? More search can improve answers but increases cost.

BFS vs DFS: Memory Growth

BFS explores level by level, finding the shortest path.

BFS visits everything one step away before moving further. On the graph $A \rightarrow B, C \rightarrow D, E, F$ the traversal order is A, B, C, D, E, F. It uses a Queue (FIFO), first in, first served.

1 Shortest path guarantee

The first time BFS reaches a target node, it has used the fewest hops by construction. No shorter route exists because all closer nodes were already visited first.

2 Memory grows with the frontier

Level 1 may hold 10 nodes, level 2 holds 100, level 3 holds 1,000. The queue stores the entire frontier at once, so memory usage grows rapidly as the graph widens.

BFS is the right choice when the answer is nearby, but the memory cost grows with every level explored.

DFS chooses one path and follows it completely before backtracking.

Unlike BFS, Depth-First Search dives all the way down one branch before it considers any other. It uses a Stack (LIFO), last in, first out, or recursion. The result is a traversal that goes deep before it goes wide.

1 Traversal order: $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F$

DFS visits A, then immediately dives to B, then D, then E, fully exhausting that branch, before backtracking to C and then F. It never holds a whole frontier in memory, only the current path.

2 Stack (LIFO) — memory-light, but may wander

Advantage: only the current path is stored, so memory usage is far smaller than BFS. Drawback: if the goal sits next to the root but behind a huge branch, DFS may explore that entire branch first — taking much longer even when the answer is close.

Think of DFS as stacking plates, the last plate placed is the first one removed.

Examples

Three AI Examples of Graph Traversal

1 Agent Planning — Vacation Booking

Flight A → Hotel A or B; Flight B → Hotel C or D. BFS evaluates many plans before committing; DFS picks one branch and explores it fully before backtracking.

2 Multi-Hop RAG — Drug Interactions

Which drug interacts with the medication for the patient's condition? The AI walks: Patient → Disease → Medication → Drug Interactions — three hops across a knowledge graph.

3 Tree of Thought — Reasoning Paths

The LLM explores Reasoning Path A, B and C rather than one path — searching a graph of possible thoughts. More exploration improves answers but costs more compute.

The Exploration–Cost Trade-off

At depth 3 an agent evaluates ~20 possibilities; at depth 10 it may evaluate millions — slower, more expensive, but potentially better answers.

The PM Question

How deep or wide can the agent search before we stop? Accuracy, latency, cost and safety all depend on the answer to that single design decision.

Running Question

Every traversal step costs GPU time and money, search limits prevent infinite loops, runaway reasoning, and runaway infrastructure bills.

1.3 Algorithms

Dynamic Programming: Solve Once, Reuse Forever

Dynamic Programming: break a problem into overlapping subproblems, solve each once, store and reuse.

The core idea: Imagine repeatedly calculating 25×48 on homework.
You write the answer down once and reuse it. DP does the same for subproblems.

So how does it work? Some problems call the same subproblem many times:

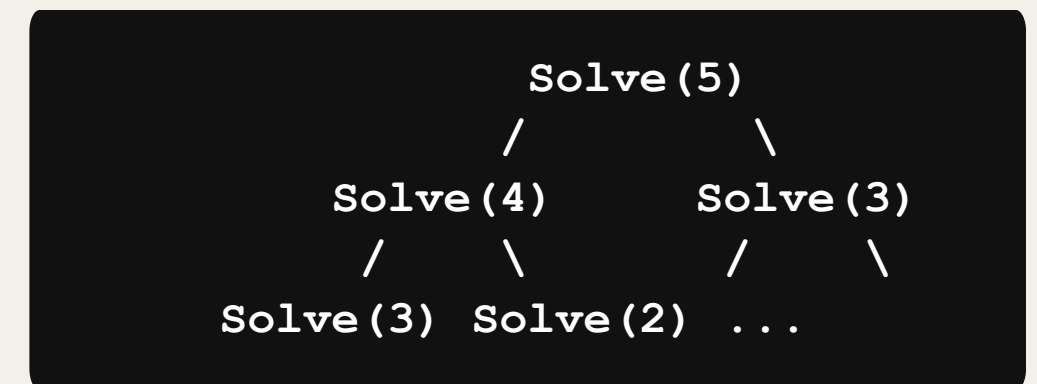
From the neighboring example:

Solve(5) calls Solve(4) and Solve(3), which both call Solve(3) and Solve(2).

DP computes Solve(3) only once, stores it in a cache, and reuses it everywhere.

This turns exponential time into polynomial time.

Canonical examples: edit distance (Levenshtein – stores intermediate comparisons so no substring pair is ever compared twice) for spell-check and fuzzy matching;
sequence alignment; knapsack; diff and fuzzy dedup of training data.



The connection that matters: the KV cache in LLMs is DP thinking, why recompute attention over all previous tokens when you can cache the Keys and Values?

1.3 Algorithms: Dynamic Programming

Memoization: Three Caches, One Idea

The stored results are called a cache or memoization table.

```
First time:
Solve(3)
↓
Compute
↓
Store Answer
```

```
Later:
Solve(3)
↓
Look in Cache
↓
Reuse Answer
```

```
Without caching:
Compute A
↓
Compute B
↓
Compute A again
↓
Compute B again
```

```
With caching:
Compute A once
↓
Store
↓
Reuse forever
```

The same principle at three levels of the stack

KV Cache in LLMs

```
Hello
↓
Hello, how
↓
Hello, how are
↓
Hello, how are you
```

```
Without Caching:
Token 4
↓
Recompute Token 1
Recompute Token 2
Recompute Token 3
```

```
With Caching:
Previous
Computations
↓
Stored in KV
Cache
↓
Reuse them
```

Prompt Prefix Caching

```
System Prompt
+
User Question
```

```
System Prompt
↓
Cache
↓
Reuse
```

Tool Calls

```
What's Apple's
stock price?
```

```
API Result
↓
Cache
↓
Reuse
```

Each cache hit:
lower latency, lower GPU usage, lower cost.

The PM question: Whenever work repeats: can we compute once and cache it?
Watch for: same prompt prefix, same retrieval, same DB query, same API call, same embeddings, same LLM response

C1.3 Algorithms

Greedy Algorithm

A Greedy Algorithm always chooses the best-looking option right now, without thinking about future consequences. It never goes back to change its decision.

The Hill Climbing Problem

```
always take the steepest step
```

Greedy climbs by always choosing the steepest path upward. It reaches a peak, but that peak may be a local hill, not the highest mountain. Once at the top of a small hill, every step looks downward, so greedy stops. The global summit remains unseen.

```
Greedy choice → local peak
Local peak    → not global max
Global max     → never reached
```

Example: 100 Rupees to Spend

```
budget = 100 rupees
maximise total value
```

Greedy picks the single highest-value item first. But combining two cheaper items beats it. The best immediate choice does not always lead to the best final result.

```
A: cost 100 → value 100 ✓ greedy
B: cost  60 → value  70
C: cost  40 → value  50
B + C      → value 120 ✓ optimal
```

The pattern: Greedy is fast and simple, and sometimes provably optimal (Huffman coding, Dijkstra). But when choices interact, locking in the best-looking option early forecloses combinations that score higher overall. Recognising when a problem has this structure is the design skill.



1.3 Algorithms: Greedy Decoding

**Greedy decoding IS a greedy algorithm.
One path. Always the highest probability.**

Given the prompt "the capital of France is":

The Capital of France example

1

Prompt: "the capital of France is"

Paris 95% → London 2% → Berlin 1% → Rome 1%

Greedy always picks Paris. Correct here — but it only ever explores this single highest-probability path.

The once upon a time trap

2

Prompt: "once upon a" — time 70%, day 20%, planet 5%. Greedy always picks time. Every story becomes "once upon a time." A richer path like "once upon a distant planet" is never explored.

Greedy decoding: maximum speed, zero exploration, one path, always the locally best token.

1.3 Algorithms: Greedy Decoding

Sampling, Beam Search, and Reasoning Models

Three ways to escape the greedy trap

Sampling

Occasionally pick a lower-probability token

- more creative output
- less repetitive
- less predictable

Controls:

Temperature

Top-k

Top-p

Beam Search

Keep K paths alive simultaneously:

Path A — score

Path B — score

Path C — score

Retain only best K at each step.

Better final text, more compute, higher latency.

Reasoning Models

Explore several reasoning paths, compare, then answer.

Path 1 — eval

Path 2 — eval

Path 3 — eval

Deliberately spends more compute to find a better solution.

= test-time compute

When to use each:

- Sampling → creative writing, chat
- Beam search → translation, summarization
- Reasoning → complex multi-step tasks

Each strategy trades speed and cost for quality and diversity.

Product decision: Greedy is fast and cheap. Sampling adds diversity. Beam search improves quality at a cost. Reasoning models spend the most compute, and deliver the best answers on hard problems.

1.3 Algorithms

Decoding Strategy Is a Product Decision

A PM decides which decoding strategy fits the product.

Strategy	Best For	Trade-off
Greedy (Temp = 0)	Deterministic answers, evaluations, customer support	Fast, cheap, but can be repetitive or suboptimal
Sampling	Creative writing, brainstorming, chat	More diverse, but less reproducible
Beam Search	Translation, summarization	Higher quality, but slower and more expensive
Reasoning/Test-Time Compute	Complex reasoning tasks	Best quality, highest latency and cost

PM Conversation:

A PM might ask: “Are we decoding greedily (temperature 0) or sampling?”

This determines:

- **Consistency:** Will identical prompts always produce the same answer?
- **Creativity:** Are responses varied or repetitive?
- **Latency:** How quickly does the model respond?
- **Cost:** How much GPU computation is required?
- **User Experience:** Is the product optimized for reliability or exploration?

Key Idea:

A Greedy Algorithm always chooses the best immediate option. In LLMs, greedy decoding always selects the highest-probability next token. It is fast and deterministic, but because it never explores alternatives, it may produce a worse overall result than methods that spend more computation searching multiple possibilities.

4

Complexity as the PM's power tool

*Two AI cases that matter most: attention and retrieval
And the chain that turns either into a number finance accepts.*

C1.4 Complexity as the PM's Power Tool

The three notations, and the two extra words

Big-O (O): Upper bound: “grows no faster than.” Worst case. This is the one everyone means in conversation.

Big- Θ (Theta): Tight bound: grows exactly at this rate; upper and lower match.

Big- Ω (Omega): Lower bound: “grows at least this fast.”

Amortized: Average over a sequence of operations, array append is $O(1)$ amortized despite occasional resizes.

Best / average / worst case: Hash lookup is $O(1)$ average and $O(n)$ worst. When someone quotes a single number, ask which one they mean.

Say “Big-O” in the room. Keep Θ and Ω for when someone is being pedantic. Nobody has ever won an architecture argument with Omega.

C1.4 Complexity as the PM’s Power Tool

The complexity ladder with feeling

Time complexity tells you how the amount of work grows as the input size (n) increases. It does not tell you the exact time in seconds.

Class	Name	n = 1,000	n = 1,000,000	If n becomes 1000x larger...
O(1)	constant	1	1	no extra work
O(log n)	logarithmic	~10	~20	barely more work
O(n)	linear	1000	1000000	1000x more work
O(n log n)	linearithmic	~1e4	~2e7	slightly worse than linear
O(n²)	quadratic	1000000	1000000000000	1,000,000x more work
O(2ⁿ)	exponential	🔥 (10³⁰⁰)	💀	Impossible

Reading it. The gap between O(n) and O(n²) at a million items is a million-fold — a 10 ms response versus a 3-hour one on identical hardware.

The counter-lesson: Big-O hides constants. For small n, an O(n²) loop can beat an O(n log n) one. Don't force an ANN index and a distributed cache for 5,000 documents and 3 QPS.

The exponential wall, concretely. n = 10 → 1,024 possibilities. n = 20 → 1 million. n = 50 → more than one quadrillion. Each new input doubles the work.

C1.4 Complexity as the PM's Power Tool

Why $O(n^2)$ is dangerous, and when to ignore it

```
O(n)    → 1,000,000 operations
O(n²)   → 1,000,000,000,000 operations
```

1 million times more work.

If $O(n)$ takes 10 ms,
 $O(n^2)$ might take hours.

```
n = 10
100n = 1000    n² = 100    ← n² wins
```

```
n = 1,000,000
100n = 100M    n² = 1T     ← n wins
```

The PM question that resolves it. “What scale are we designing for?” Good engineering is the simplest solution that comfortably handles your expected scale, while recognising when it's time to move.

Two truths, both true

1. **Choose good algorithms:** At millions of users or billions of vectors, complexity dominates. $O(n \log n)$ over $O(n^2)$ turns hours into seconds.
2. **Don't optimize too early:** With 5,000 documents and 3 searches/sec, linear search answers in milliseconds. ANN indexes and distributed caches add complexity with no user benefit.

Key Takeaway: The art is balancing two truths:

1. Algorithmic complexity determines long-term scalability.
2. Don't introduce sophisticated infrastructure before the scale justifies it.

C1.4 Complexity as the PM’s Power Tool

Attention is $O(n^2)$, the PM example

The Big Idea:

The most important operation inside a Transformer is self-attention. Self-attention lets every token look at every other token to decide what information is relevant. This is powerful, but expensive.

Example

"The cat sat on the mat."

Suppose it becomes 6 tokens.

- 1. The
- 2. cat
- 3. sat
- 4. on
- 5. the
- 6. mat

When processing "sat", the model asks:

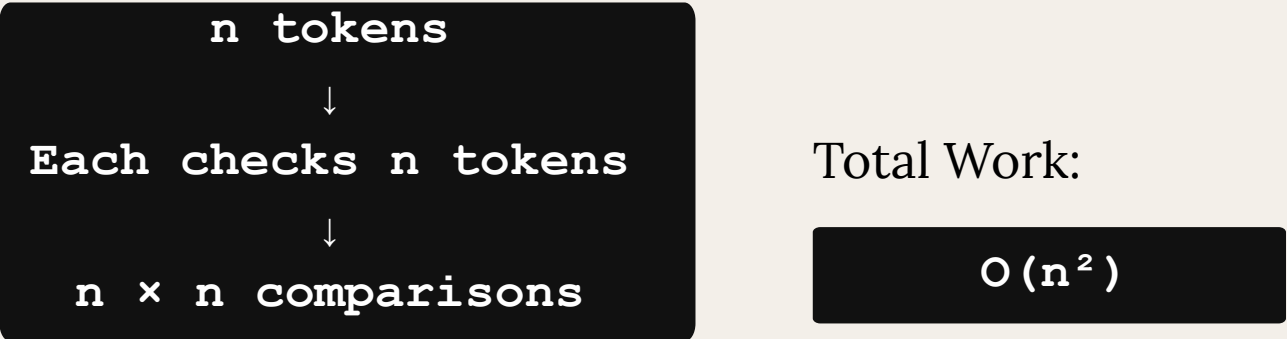
Should I pay attention to "The"?
"cat"?
"on"?
"the"?
"mat"?

It compares with every token.
Now "cat" does the same.
Then "mat" and eventually:

Every token
↓
Looks at every other token

Why $O(n^2)$?

If there are n tokens, then each token compares with n tokens.



4K → 128K tokens = 32× longer

Attention work $\approx 32^2 \approx 1000\times$

Doubling context $\approx 4\times$ the cost

C1.4 Complexity as the PM's Power Tool

Why 128K context is expensive, and why RAG exists

```
128K / 4K    =    32×
```

```
But attention is quadratic:
```

```
322    =    1024×
```

Approximately 1000 times more attention computation.

1. **Early LLMs were small:** 2K and 4K windows, because larger contexts became too expensive too fast.
2. **Long context costs more:** Higher GPU usage, more memory, longer response times.

Why not put everything into the prompt?

```
100,000 documents in the prompt  
→ quadratic cost explosion
```

```
Instead — RAG:
```

```
User Question
```

```
↓
```

```
Retrieve Top 5 Relevant Documents
```

```
↓
```

```
Only those go into the prompt
```

The result. The model processes only the relevant information. Faster, cheaper, and often more accurate than stuffing the corpus into context.

C1.4 Complexity as the PM's Power Tool

The efficiency research, and the honest caveat

"Can we avoid comparing every token with every other token?"

- **FlashAttention:** Computes the same attention more efficiently, reducing memory traffic and improving GPU utilization.
- **Sliding window / sparse attention:** Each token attends only to nearby or selected tokens instead of all tokens.
- **State Space Models (Mamba):** Replace full attention with architectures whose computation grows roughly linearly with sequence length.

The caveat. Total inference time also includes feed-forward layers, KV cache reads and writes, memory bandwidth, and kernel optimizations.

Doubling context does not always make wall-clock exactly 4× larger.

They all attack one problem. Quadratic attention becomes expensive for long sequences. You don't need the tricks, you need to know what they're all attacking.

PM questions:

- Can RAG retrieve only what's relevant?
- Does the use case truly require 128K?
- How much extra latency and GPU cost?
- Is long-context support worth the quality gain?

C1.4 Complexity as the PM's Power Tool

Why brute-force vector search doesn't scale

Imagine you have a library with 100 million books. A user asks: “**Find books most similar to this one.**”

There are two ways to do it:

1. Compare against every book. (Brute force)
2. Search a smart index that narrows the candidates first. (ANN)

Lets do Approach-1

1. Before searching, every document is converted into an embedding.

```
"What is AI?"  
↓  
[0.23, -0.41, 1.12, ...]  
  
1536 dimensions per  
document
```

2. Suppose:
 - a. 100 million documents
 - b. Each embedding has 1536 numbers

```
Query  
↓  
Document 1 → Document 2  
→ Document 3 → ...  
↓  
Document 100,000,000
```

3. Lets do the math

```
n = 100,000,000 documents  
d = 1,536 dimensions  
(multiplications +  
additions)  
≈ 1.5 × 1011 multiply-  
adds PER QUERY growing  
with users
```

With embedding size fixed at 1536, the dominant factor in $O(n.d)$ is $O(n)$ (The no. of documents. Double the documents, double the work.

The wall is predictable.
Hundreds of ms to seconds per user, growing linearly with your corpus and multiplied by QPS.

PM takeaway: Brute force is fine until it isn't: fine at ~1M vectors and low QPS, a wall at 100M and high QPS. The shape of the wall is predictable, so you can forecast when you'll hit it.

C1.4 Complexity as the PM's Power Tool

ANN (Approx Nearest Neighbors): Build an index, check a few hundred

Lets do Approach-2

Instead of searching everything: build an index first. (Think of Google. When you search, it doesn't read every webpage. It uses an index. ANN does the same for embeddings.)

```
Query
↓
ANN Index
↓
Small Candidate Set
↓
Best Matches
```

Instead of 100 million vectors, it may check only a few hundred or a few thousand.

Common ANN Methods:

- **HNSW**: Creates a graph connecting nearby vectors; the search walks the graph. Like navigating a city using roads instead of flying over every house.
- **IVF (Inverted File Index)**: Groups vectors into clusters. A query finds the nearest cluster and searches only inside it.
- **Product Quantization (PQ)**: Compresses vectors → less memory, faster comparisons, slight loss in precision.

The trade-off. ANN is fast because it is approximate. The true top result may occasionally be replaced by one that is almost as good, usually worth it.

C1.4 Complexity as the PM's Power Tool

Recall vs latency vs cost: pick your corner

Every vector database balances three competing goals

Recall

Did we find the actual best documents?

Higher recall = better search quality.

Latency

How quickly can we answer?

Lower latency = happier users.

Cost

How much CPU, GPU and memory?

Lower cost = cheaper infrastructure.

You usually can't maximize all three. Improving one often requires sacrificing another.
That is the entire vector-database conversation.

PM Thinking: Imagine two products.

Startup: 500,000 documents, 5 searches/sec.
Brute force is perfectly acceptable → simple system, low engineering effort.

Enterprise: 100 million documents, 10,000 searches/sec. Brute force is prohibitively expensive; ANN becomes essential.

The Scaling Wall: Notice the pattern.

```
More Documents
↓
Every Search Becomes Slower
↓
More Users
↓
Total Work Explodes
```

The total work is roughly proportional to:

```
Total work ≈ Corpus Size × QPS

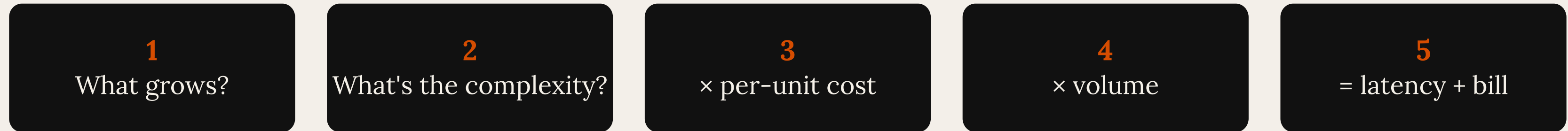
Docs 10× and Traffic 10×
→ 100× more search work
```

PM asks: when does our corpus outgrow brute force? Is 99% recall acceptable if it cuts latency 10×? When does an ANN index turn ROI-positive?

C1.4 Complexity as the PM's Power Tool

Complexity → latency → cost, in five steps

A rough but powerful back-of-envelope chain:



Run this in the meeting. “Each request retrieves top-20 chunks (~2K tokens) + 1K prompt + 500 output ≈ 3.5K tokens. At \$X per 1K tokens × 1M requests/day = \$Y/day. Switch to long-context stuffing at 50K tokens/request and tokens go ~14× — so the bill goes ~14× and latency rises. That's why we RAG.”

You just turned a data-structure fact into a defended architecture decision in front of finance. That is the PM job.

C1.4 Complexity as the PM’s Power Tool: Complexity → latency → cost, in five steps

Step 1–4: from “what grows” to a cloud bill

Step 1: What is Growing?

Every AI system has something that grows over time.

Metric	What grows?
Users	Daily Active Users
Documents	Knowledge base size
Tokens	Prompt length
Tool Calls	Agent actions
QPS	Queries per second

PM Ask: "What variable will become 10× larger in the next year?"

Step 2: What is the Complexity?

Every AI system has something that grows over time.

Operation	Complexity
Reading a cache	O(1)
Linear search	O(n)
Sorting	O(n log n)
Attention	O(n²)
Exhaustive planning	O(2 ⁿ)

Step 3: Convert Work into Time or Money

Every operation has a cost.

- milliseconds of CPU/GPU time
- GPU memory
- API cost per token
- dollars per GPU-hour

Step 4: Multiply by Volume

- One request rarely matters. A million requests do.

```
Step 3  1,000 tokens = $0.01

Step 4  1 request → 3,500 tokens
        1,000,000 requests/day

        = 3.5 billion tokens/day
```

C1.4 Complexity as the PM's Power Tool: Complexity → latency → cost, in five steps

Step 5: RAG vs long context, priced

Step 5: Make a Product Decision

Now compare two designs.

Option A — RAG

```
Retrieve 20 chunks    ≈ 2,000 tokens
+ Prompt              = 1,000 tokens
+ Response            =   500 tokens
```

```
Total = 3,500 tokens/request
```

Option B — stuff the prompt

```
Entire Knowledge Base
≈ 50,000 tokens
```

```
Total = 50,000 tokens/request
50,000 ÷ 3,500 ≈ 14× more tokens than RAG
```

Imagine your finance team asks: "Why are we investing in a vector database?"

Weak vs strong answer.

Weak: "Because everyone uses RAG."

Strong: "RAG cuts the average prompt from ~50K to ~3.5K tokens."

At 1M requests/day that's ~14× less token usage, directly reducing inference cost and latency."

C1.4 Complexity as the PM's Power Tool

Two forecasts you can make before any benchmark

Attention

Suppose you increase context:

```
Context 4K → 128K    = 32×  
Attention ≈ O(n2)
```

```
Work ≈ 322 ≈ 1000× (Attention work grows)
```

```
→ longer responses, more GPU memory
```

Vector search

Suppose your document corpus grows:

```
Corpus 1M → 100M    = 100×  
Traffic 100 → 1000 QPS = 10×
```

```
Combined ≈ 100 × 10 = 1000×
```

```
→ now you know when ANN pays
```

The PM Thinking Pattern:

Whenever someone proposes a feature, mentally walk through this chain:

What grows? → How does computation grow? → How much compute is that? → How much latency? → How much does it cost? → Is there a better architecture?

Stop asking: “Can we support 128K context?”

Start asking: What's the average prompt length? How many extra tokens does this add? Does this turn an O(n) operation into O(n²)? What's monthly bill at 10× traffic? Could retrieval or caching reach the same outcome more efficiently?

The key takeaway. The highest-leverage skill isn't memorizing algorithm names, it's translating algorithmic complexity into business consequences.

Growth → Complexity → Compute → Latency → Cost → Product Decision.

C1.4 Complexity as the PM's Power Tool

The questions that do the work

Complexity comes first: “Why is this slow?” usually has a Big-O answer before it has an infra answer. Adding GPUs to an $O(n^2)$ algorithm is pouring money into a leak.

Precompute / index / cache = moving cost from query-time to build-time, the oldest trade in systems. Then ask about staleness: a cache is a copy, and when the source changes the copy is wrong until rebuilt.

Averages lie; tails matter. A great average with an ugly p99 feels broken to the unlucky 1% — and at 1M requests/day that's 10,000 angry users daily.

The ready-made set

- What grows here, and how does cost grow with it?
- Where does this break — 10×? 100×? What melts first?
- What are we trading — memory for latency? freshness for speed? recall for cost?
- Real scale risk, or a scale we won't see this year?
- What's the p95/p99, not just the average?

5

The engineering negotiation

Under-building and over-building are both failures.

You are the calibration.

C1.5 The Engineering Negotiation

Two archetypes, two scripts

Archetype 1: “That's $O(n^2)$, it won't scale.”

Is the alarm real?

$O(n^2)$ in what? Does that quantity actually grow? If n is “number of form fields,” n^2 is nothing. If n is “documents in the corpus,” it's everything.

What n today, at launch, at 12 months? Where is the first wall?

What's the fix's cost? Build time, memory, a new dependency, staleness, and is it worth it now?

Principal move. “Ship the $O(n^2)$ version at $n=5,000$ and revisit at $n=500,000$, write the trigger into the doc.”

Archetype 2: “We need an index / cache / distributed system.”

Is it premature?

What does it buy at our current scale? Measured, not theoretical.

What does it cost us? Build time, ongoing ops, a new failure mode, cache-invalidation and staleness bugs.

What breaks if we don't do it? For two more quarters.

Principal move. Protect the team from gold-plating as fiercely as from under-building.

6

Interview prep

Six questions.

Named examples and numbers, not vocabulary.

C1.6 Interview Prep

The three you must answer cold

Q1 · Why does an $O(n^2)$ algorithm concern you at scale?

Cost grows with the square of the input, 10× the input is 100× the work. Great in a demo, dead in production. Self-attention is the canonical AI example: 4K → 128K context is ~1000× the attention compute, which is why long context is expensive and slow and why RAG is often cheaper. But I'd also ask what n is, quadratic in a small bounded quantity is a non-issue.

Q2 · What's a hash table and why is it everywhere in AI infra?

Maps keys to values in $\sim O(1)$ average by hashing straight to a slot. It's the backbone of the KV cache, prompt/response caches, feature stores, and deduplication. Whenever cost drops because “we cached it,” a hash table is underneath, and the metric I'd watch is cache hit rate..

Q3 · Your engineer proposes brute-force vector search for v1. Good or bad?

Usually “good, for now.” $O(n \cdot d)$ per query is fine under ~1M vectors at modest QPS and avoids build cost, memory overhead, and recall loss. Ship it with a documented tripwire — revisit at ~1M vectors or when p95 exceeds budget — then move to HNSW or IVF.

C1.6 Interview Prep

The executive question and the two stretches

Q4 · Explain caching to an executive.

Paying to compute an answer once and reusing it many times instead of recomputing every request, trading a little memory, and the risk the copy goes stale, for big wins in speed and cost. If 40% of requests hit cache, that's 40% fewer expensive model calls. The catch is freshness: a cache is a copy, so we need an invalidation rule.

Q5 (stretch) · Walk me through the latency of a single RAG request.

Embed the query, retrieve top-k from the vector index (~tens of ms with ANN), optionally re-rank the shortlist, assemble the prompt, then the LLM call, which splits into prefill (roughly linear in input tokens, dominated by $O(n^2)$ attention on long inputs) and decode (one token at a time, using the KV cache). Output generation and any re-ranker drive the tail; queue-wait is the hidden one that only shows under load.

Q6 (stretch) · Why are GPUs fast for AI, in data-structure terms?

The core operation is dense matrix multiplication over contiguous arrays, thousands of independent dot products, and GPUs pair thousands of cores with memory designed for coalesced access. Deep learning uses dense tensors rather than pointer-based structures precisely to keep the GPU fed.

Track-C · Part-1

**What grows,
and how does cost grow with it?**

Ask it relentlessly. That question is 80% of the value of this track.